

AI Tools for Economics Research

Session 2 — Worked examples

Friday 29 May 2026

10:30–12:00 · 14:30–16:00

Banco de Portugal

João B. Sousa · Nova SBE

TODAY

Real tasks, live.

Four examples in identical structure: the task, the prompt, what comes back, what remains to verify.

THE FOUR EXAMPLES

What we'll cover today.

Literature triage a folder of PDFs → comparison table.

Data cleaning two raw series → aligned panel, summary, plot.

Code review a script with a planted bug.

Replication clone a repo, reproduce a figure.

Live demonstration.

A session lives in one folder.

```
$ cd ~/my-project      # a folder under version control
$ claude               # start a session here

> /help                # list everything you can type
> Esc                  # interrupt the agent mid-task
> /clear               # forget the conversation, keep the session

> /exit                # leave    (or Ctrl-D)

$ claude --continue    # reopen the last session, context intact
```

Start it where your project is —
never in your home directory or a
folder holding credentials or
supervisory data.

STANDING INSTRUCTIONS

CLAUDE.md — read at the top of every session.

Recall Monday's context bundle:

`CLAUDE.md` sits in the preamble,
prepended before your first prompt.

Write a convention once; the agent
follows it every turn, every session —
no need to repeat it in prompts.

Two levels, both loaded
automatically. Add a line mid-
session by typing it with a leading

`#`

```
~/.claude/CLAUDE.md  ← user
                        every project, this machine
                        "Prefer Stata for empirics,
                        JAX/NumPy in Python."
./CLAUDE.md           ← project
                        shared via git with the repo
                        "data/raw/ is read-only;
                        write to processed/."
```

01

Literature triage.

THE TASK

Thirty PDFs, one afternoon.

Skim each paper, summarise its data and method, cluster by approach, isolate the subset matching your question, build a comparison table.

Routine work at the start of a project. The agent handles the first pass; verification remains yours.

Six papers on inflation expectations.

```
papers/inflation_expectations/  
├─ cagan_1956_hyperinflation.pdf  
├─ muth_1961_rational_expectations.pdf  
├─ friedman_1968_role_of_monetary_policy.pdf  
├─ coibion_gorodnichenko_2015_info_rigidity.pdf  
├─ armantier_etal_2017_sce_overview.pdf  
└─ candia_coibion_gorodnichenko_2021_firms.pdf
```

Six today — enough to watch the workflow live.
The same prompt scales to thirty without
change.

THE PROMPT

What we type.

Read every PDF in papers/inflation_expectations/.

For each one, write:

- title, year, authors
- data source and sample
- identification or empirical strategy
- headline result, one sentence

Then a markdown comparison table with the same columns.

For the headline column, quote the sentence in the paper that supports it, with a page number. Do not paraphrase.

Two design choices: a specified output format, and a quote-with-page rule to constrain fabrication.

WHAT COMES BACK

A table you can audit.

A markdown table: one row per paper, the requested columns, the headline column populated with quoted sentences and page numbers.

The quote-with-page rule anchors verification. Without it, the headline column reflects the model's prior rather than the paper's content.

Paper	Data	I
Cagan	hyperinf.	a
Muth	theory	r
Friedman	review	e
Coibion & G.	SPF + cons.	I
Armantier et al.	SCE	s
Candia et al.	firm survey	n

BEFORE ANY OF IT REACHES A DRAFT

The output is fluent. Verify it.

Fabricated quotes a headline sentence that is not in the paper.

Wrong page numbers right quote, wrong location — worse on scanned PDFs.

Conflated papers a result attributed to the wrong row.

Open the cited page before you trust any cell. Failure mechanisms in Session 3.

02

Data cleaning and summary tables.

THE TASK

Two series — one on disk, one to fetch.

US CPI from Monday is already in `data/raw/`.

The Portuguese HICP is not — the agent retrieves it from Eurostat, then aligns the two and compares.

Obtaining the data is the first step. The agent does it the way you would: find the series, download it, save it to the repo.

THE RAW MATERIAL

We start with one file.

```
data/raw/  
└─ cpi.csv      FRED, US CPI-U, monthly, 1947–2026
```

The Portuguese HICP is not on disk. The agent downloads it from Eurostat, then reconciles two different date formats, column names, and missing-value codes.

What we type.

US CPI is on disk: data/raw/cpi.csv (FRED).

1. Download monthly HICP for Portugal (all-items) from Eurostat – just that one series, plain CSV. Save to data/raw/hicp_pt.csv.
2. Align both to monthly frequency on the common sample. Compute year-over-year inflation.
3. Write:
 - data/processed/inflation_us_pt.csv long format
 - a markdown table: mean, sd, min, max by decade
 - slides/assets/inflation_us_pt.png two lines

Stop and ask if anything is ambiguous before writing.

WHAT COMES BACK

Three outputs.

A CSV in long format that `read_csv` can parse.

A summary table ready to paste into a memo.

A PNG ready to drop into a slide.

Diffs are stored in git; commands in shell history. The session is reproducible.

Decade	US mean	PT mean
1990s	2.0	2.1
2000s	2.6	2.6
2010s	1.8	1.2
2020s	4.0	3.2

Four checks before the table goes into a memo.

Column choice headline "CPI" vs core
"CPILFESL" — whichever you intended.

Types and missingness spot-check the date
column and each series' missing-value count.

Date conventions FRED stamps first-of-month,
Eurostat last-of-period — check they align.

Sample period was "common sample" read as
intersection or forward-fill?

What the two examples share.

1. The agent works on your filesystem with one prompt — no copy-paste, no upload limits.
2. Structured prompts (specified format, quote-with-page rule, stop-and-ask) yield outputs ready for audit.
3. Every step is reproducible: edits land in git, commands in shell history.

After lunch: code review, and a replication exercise.

BREAK

14:30

Back after lunch.

03 · AFTERNOON

Code review and debugging.

THE SETUP

A script you have not read line by line.

It runs without error and writes a CSV. The plot looks plausible at a glance. You want to reuse it.

Code that runs and produces plausible output is easy to leave unchecked.

THE CODE

code/scripts/compute_inflation.py

```
import pandas as pd

cpi = pd.read_csv("data/raw/cpi.csv", comment="#",
                  parse_dates=["observation_date"])
cpi = cpi.sort_values("observation_date")

# year-over-year inflation
p = cpi["CPIAUCSL"]
cpi["yoy"] = 100 * (p / p.shift(11) - 1)

cpi[["observation_date", "yoy"]].to_csv(
    "data/processed/inflation.csv", index=False)
```

22

Twelve lines, one bug. Identify it
before the next slide.

THE PROMPT

What we type.

```
Walk me through code/scripts/compute_inflation.py line by line. What does each step do? Flag anything you would double-check against a published source before trusting the output. Do not edit the file.
```

"Do not edit" is the constraint that matters. It keeps the walkthrough and any subsequent fix in separate diffs.

WHAT COMES BACK

A line-by-line walkthrough — and a flag.

The agent describes each step, then notes one line worth double-checking: `shift(11)` compares each month to **eleven** months earlier, but year-over-year on monthly data requires `shift(12)`.

The fix is one character. Without the walkthrough, nothing else in the script would have flagged it.

Asking for an explanation surfaces issues alongside the description; asking only for a fix often returns style edits.

WHAT YOU DO NEXT

Three steps before the fix lands.

Read the explanation against the code the walkthrough is the audit trail.

Confirm the symptom compare a few output values against a trusted source (FRED, BdP).

Ask for the diff before applying one-character fixes still go through the same review loop.

Each step happens before the fix is written to the file.

THE HABIT

Diagnose first, fix second.

Two turns produce two diffs and two opportunities to review before any change is written.

04

Replication.

THE TASK

Clone a repo. Reproduce one figure.

The exercise required when you need to reproduce a published figure — for a referee report, a robustness check, or a presentation.

Replication packages are typically written once and not revisited. Environmental drift accumulates by the time anyone clones them.

WHAT USUALLY BREAKS

Three things that commonly break.

Package versions a dependency pinned to an old version; the current one renamed a function, and the script raises.

Hard-coded paths absolute paths to the author's machine, written into the scripts.

Missing data listed as "available on request," with no working way to obtain it.

Model-based — no data to download, so the third never arises.

THE PROMPT

What we type.

```
Clone https://github.com/shade-econ/sequence-jacobian  
into ./replication/.
```

```
Read the README. Set up the environment. Run  
notebooks/hank.ipynb and reproduce the impulse responses  
to a monetary policy shock. Save the figure to ./figures/.  
If a script (not the notebook) makes the figure, save it  
to ./code/ for inspection. Stop after the figure is saved.
```

```
For every change you make to the code or environment, log  
it as a bullet so I can read the list before you continue.
```

The figure and its code land on
disk — to inspect and re-run.

WHAT TO ASK FOR

Ask for outputs you can audit and re-run.

The result the figure or table, to compare with the published one.

A change-log every code and environment edit, to see what changed.

The environment the deps and commands, to rebuild from a clean clone.

Name the deliverables — you get what you ask for.

WHAT YOU STILL VERIFY

Match the reproduction to the paper.

Same axes, same scale, same legend.

A reference number market-clearing residuals,
a steady-state value — confirm it matches the
notebook.

The change-log environment only; the model
code is never touched.

If a change touches the model —
calibration, equations, the
solution method — stop.
Environment fixes should not
alter the results.

05

Q&A.

Three habits to take forward.

1. **The agent compresses the mechanical parts of research** — triage, cleaning, replication setup, reading unfamiliar code.
2. **Structured prompts are the lever** — specify the format, require citations, ask the agent to stop and explain.
3. **Diagnose first, fix second** — review the diff before any change is committed.

SUGGESTED VIDEOS

Claude Code for Applied Economists.

Paul Goldsmith-Pinkham's video series at Markus' Academy — full empirical workflows in Claude Code, from data analysis to web scraping. youtube.com/@markus_academy

See you on 22 June.

joao.sousa@novasbe.pt